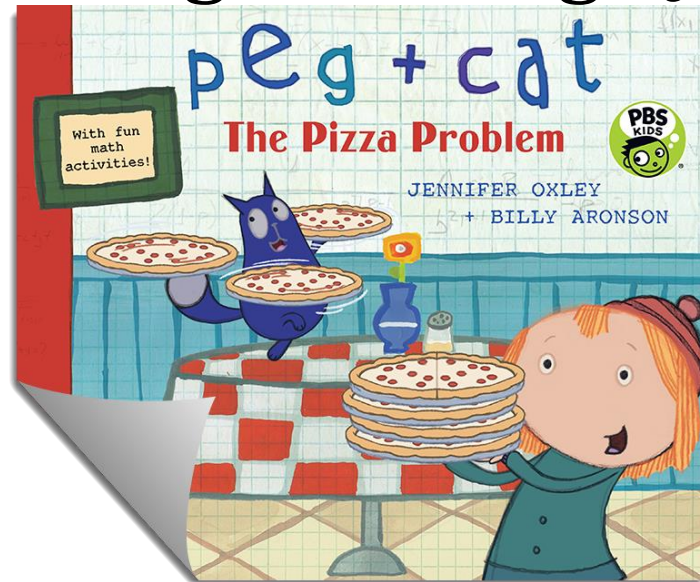


# CS 170 Programming Quiz 9



For **PQ09 - Arrays, Lists & Loops** you'll write one method that use arrays or array lists. All of the problems will use loops. This exam covers Chapter 7 in your textbook.

For this exam you'll need to know how to:

- Process arrays and lists with loops (counting, summing, extreme values and separators)

**You will not need to use partially filled arrays or 2D arrays on this exam.** As on the previous exam, your method will be tested for syntax correctness (name, public, return type and parameters) and for correctness. Work step-by-step so that you get partial credit. You can find problems similar to this on Codingbat and on Practice It.

**Note:** please do not just memorize the answers to these problems. The actual problems on the exams are always variations of the problems from the problem set.

## 1 THE ADD LESS PROBLEM

---

Start with 1 int array nums and an int min. Your method should sum all numbers in nums that are less than min. Some examples:

- `addLess({3, 5, 7, 9}, 3) → 0`
- `addLess({3, 5, 7, 9}, 9) → 15`
- `addLess({3, 5, 7, 9}, 10) → 24`

## 2 THE AVERAGE ODDS PROBLEM

---

Start with a int array named nums. Return the average of all the odd numbers in the array. A number (n) is odd if  $n \% 2 == 1$ . If there are no odd numbers in the array, return 0.

- `averageOdds({1, 2, 3})` → 1.5
- `averageOdds({2, 4})` → 0.0
- `averageOdds({2, 3, 4})` → 3.0

## 3 THE COUNT LIKE PROBLEM

---

Start with 1 String array wordList and one String word. Your method should calculate and return the number of times that word appears in the array wordList.

- `countLike({"a", "a", "b", "c"}, "a")` → 2
- `countLike({"a", "a", "b", "c"}, "c")` → 1
- `countLike({"a", "a", "b", "c"}, "3")` → 0

## 4 THE AVERAGE EVENS PROBLEM

---

Start with an int array named nums. Return the average of all the even numbers in the array. A number (n) is even if  $n \% 2 == 0$ . If there are no even numbers in the array, return -1.

- `averageEvens({1, 2, 3})` → 2.0
- `averageEvens({1, 5})` → -1.0
- `averageEvens({2, 3, 4})` → 3.0

## 5 THE SUM X PROBLEM

---

Return the sum of the extreme (largest and smallest) values in the int array nums. If there are no values in the array, then return -1.

- `sumX({9, 5, 1, 3, 7})` → 10 // (9 + 1)
- `sumX({5})` → 10 // (5 + 5)
- `sumX({})` → -1

## 6 THE COUNT DIFF PROBLEM

---

Start with 1 String array wordList and one String word. Your method should calculate and return the number of words that are different from word in the array wordList.

- `countDiff({"a", "a", "b", "c"}, "a")` → 2 // (b and c)
- `countDiff({"a", "a", "b", "c"}, "c")` → 3 // (a, a, b)
- `countDiff({"a", "a", "b", "c"}, "3")` → 4

## 7 THE MARK4 PROBLEM

---

Write a method that takes an ArrayList of Strings that places a string of four asterisks "\*\*\*\*" in front of every string of length 4. Return the modified ArrayList.

- `mark4(["this", "is" "fun"]) → ["****", "this", "is" "fun"]`
- `mark4(["lots", "of", "Java"]) → ["****", "lots", "of", "****", "Java"]`
- `mark4(["another", "example"]) → ["another", "example"]`

## 8 THE MAX LEN PROBLEM

---

Write a method `maxLen` that takes an ArrayList of Strings as a parameter and that returns the length of the longest string in the list. If your method is passed an empty list, it should return -1.

- `maxLen(["to", "be", "or", "not", "to", "be"]) → 3`
- `maxLen(["biggest", "next", "not"]) → 7`
- `maxLen([]) → -1`

## 9 THE SWITCH PAIRS PROBLEM

---

Write a method that switches the order of values in an ArrayList of Strings in a pairwise fashion. Your method should switch the order of the first two values, then switch the order of the next two, and so on. If there the list size is odd then the final element is not moved. Return the modified list.

- `switchPairs(["a", "b", "c", "d"]) → ["b", "a", "d", "c"]`
- `switchPairs(["a", "b", "c"]) → ["b", "a", "c"]`
- `switchPairs(["a"]) → ["a"]`

## 10 THE RANGE PROBLEM

---

Write a method that takes an ArrayList of int and that returns the range of values in the list; the range is 1 + the difference between the largest and smallest elements. An empty list has a range of 0.

- `range([18, 14, 29, 12, 7, 25]) → 23`
- `range([9, 9, 9]) → 1`
- `range([]) → 0`

## 11 THE STUTTER PROBLEM

---

Write a method that takes an ArrayList of Strings and that replaces every string with two of that string. Return the changed list.

- `stutter(["how", "are", "you?"]) → [how, how, are, are, you?, you?]`
- `stutter(["1", "4", "3"]) → [1, 1, 4, 4, 3, 3]`
- `stutter([]) → []`

## 12 THE MERGE TWO PROBLEM

---

Start with two ArrayLists of String, A and B, each with its elements in alphabetical order and without any duplicates. Return a new list containing the first N elements from the two lists. The result list should be in alphabetical order and without duplicates. A and B will both have a size which is N or more. Your solution should make a single pass over A and B, taking advantage of the fact that they are in alphabetical order, copying elements directly to the new list.

- `mergeTwo(["a", "c", "z"], ["b", "f", "z"], 3) → ["a", "b", "c"]`
- `mergeTwo(["a", "c", "z"], ["c", "f", "z"], 3) → ["a", "c", "f"]`
- `mergeTwo(["f", "g", "z"], ["c", "f", "g"], 3) → ["c", "f", "g"]`

## 13 THE SCORE UP PROBLEM

---

Start with two ArrayLists of String: "key" containing the correct answers to an exam, like ["a", "a", "b", "b"], and "answers" contains a student's answers, with "?" representing a question left blank. The two lists are not empty and are the same length. Return the score for this list of answers, giving +4 for each correct answer, -1 for each incorrect answer, and +0 for each blank answer.

- `scoreUp(["a", "a", "b", "b"], ["a", "c", "b", "c"]) → 6`
- `scoreUp(["a", "a", "b", "b"], ["a", "a", "b", "c"]) → 11`
- `scoreUp(["a", "a", "b", "b"], ["a", "a", "b", "b"]) → 16`

## 14 THE WORDS WITHOUT PROBLEM

---

Given an ArrayList of String, return a new list where all the strings of the given length are omitted. Do not modify the method parameter.

- `wordsWithout(["a", "bb", "b", "ccc"], 1) → ["bb", "ccc"]`
- `wordsWithout(["a", "bb", "b", "ccc"], 3) → ["a", "bb", "b"]`
- `wordsWithout(["a", "bb", "b", "ccc"], 4) → ["a", "bb", "b", "ccc"]`

## 15 THE COPY EVENS PROBLEM

---

Given an ArrayList of positive integers, return a new list of length "count" containing the first even numbers from the original list. The original will contain at least "count" even numbers.

- `copyEvens([3, 2, 4, 5, 8], 2) → [2, 4]`
- `copyEvens([3, 2, 4, 5, 8], 3) → [2, 4, 8]`
- `copyEvens([6, 1, 2, 4, 5, 8], 3) → [6, 2, 4]`

## 16 THE COPY ENDY PROBLEM

---

We'll say that a positive int n is "endy" if it is in the range 0..10 or 90..100 (inclusive). Given an ArrayList of positive Integers, return a new ArrayList of size "count" containing the first endy numbers from the original list. An easy way to do this is to create a separate `isEndy(int n)` method to test if a number is endy. The original list will contain at least "count" endy numbers.

- `copyEndy([9, 11, 90, 22, 6], 2) → [9, 90]`
- `copyEndy([9, 11, 90, 22, 6], 3) → [9, 90, 6]`
- `copyEndy([12, 1, 1, 13, 0, 20], 2) → [1, 1]`